

Algorithms in CompBio – Sequence Alignment

Task 1: Implement the following similarity based dynamic programming algorithm for global alignment:

DP Problem:

$$T(i, j) = \begin{cases} T(i-1, j) - d & \text{gap}(y) \\ T(i, j-1) - d & \text{gap}(x) \\ T(i-1, j-1) + s(x[i], y[j]) & \text{match} \end{cases}$$

where...

d = linear penalty for gap

s(x[i], y[j]) = similarity between x[i] and y[j] defined by **substitution matrix**

****code is found in zipfile also attached****

Questions:

1. describe the algorithm for aligning three sequences

The DP table becomes three-dimensional instead of two-dimensional, with each dimension representing a sequence. The recurrence relation now accounts for matches and gaps across three sequences

$$T(i, j, k) = \begin{cases} T(i-1, j, k) - d & \text{gap}(y) \\ T(i, j-1, k) - d & \text{gap}(x) \\ T(i-1, j-1, k) + s(x[i], y[j]) & \text{gap}(z) \text{match}(x, y) \\ T(i-1, j, k-1) + s(x[i], z[k]) & \text{gap}(y) \text{match}(x, z) \\ T(i, j-1, k-1) + s(y[j], z[k]) & \text{gap}(x) \text{match}(y, z) \\ T(i-1, j-1, k-1) + s(y[j], z[k]) + s(x[i], z[k]) + s(x[i], y[j]) & \text{match}(x, y, z) \end{cases}$$

One way to still implement this in the 2d case is to use pointers

2. does your algorithm scale well to p sequences?

My algorithm does NOT scale well to p sequences. This is because the memory and computational complexity of my algorithm grows roughly **exponentially** with the number of sequences because the DP table requires $O(L^p)$ space for p sequences of length L

3. Do you have any ideas from the class on how to speed up the alignment of 2 sequences using techniques we have discussed in class? multiple sequences? It does not need to be an optimal alignment.

One way to implement multiple sequences could be to align two sequences FIRST and then iteratively add the others. I don't think this would be optimal because it would already align the first two without even seeing the next sequences, so the approach is rather greedy.

In my last CS class (CSE3600), I learned about parallel computing, which we could use here. We could somehow use parallel computing to process independent segments or calculate multiple pairwise alignments simultaneously, which would give more of an optimal solution.

Task 2: Implement the following similarity-based dynamic programming algorithm for local alignment.

$$T(i, j) = \begin{cases} 0 & \text{start} \\ T(i-1, j) - d & \text{gap}(y) \\ T(i, j-1) - d & \text{gap}(x) \\ T(i-1, j-1) + s(x[i], y[j]) & \text{match} \end{cases}$$

****code found in zipfile****

Task 3: Long genes are difficult to align using the standard Needleman-Wunsch algorithm. For example, aligning APOL3 and APOL4 using 1 Byte to represent each cell would yield a DP table of $> 3.2GB$. In order to align these two sequences, you are tasked with writing a program that performs global alignment using a **divide and conquer** algorithm for linear space sequence alignment. It should take as input sequences x and y and should use no more than $O(n)$ space. Use your algorithm to align the APOL3 and APOL4 gene sequences which we uploaded to HuskyCT. Start with a 5×5 similarity matrix for DNA, a 1 for a match, and 0 for gaps or substitutions. You should modify these entries to see how it affects the alignment. Write a convincing argument supported by benchmarking that your algorithm indeed uses linear space.

**** code found in zipfile****

Divide and Conquer Explanation:

to split the problem: I computed the last row of the DP table for aligning the FIRST half of seq1 with seq2 (forward computation).

I then computed the last row of the DP table for aligning the second half of seq1 with seq2, but in REVERSE (backward computation).

I then add the two rows element-wise. The position of the maximum value in this combined row indicates the optimal split point in seq2

Each subproblem is solved **recursively** using the same divide and conquer strategy

When a sequence becomes short (**base case**), we use a standard alignment algorithm (like Needleman-Wunsch) to compute the alignment directly

The alignments of the subproblems are COMBINED to form the alignment of the original sequences

results: APOL3 and APOL4 aligned with a length of 28456 with a memory size of about 6756 bytes (significantly lower than 3.2 GB)

It took about 7 minutes for the full program to run.

I used the memory profiler library to get the memory usage of the algorithm and compared it to the sequence length of each sequence to get the following benchmark (on the next page):

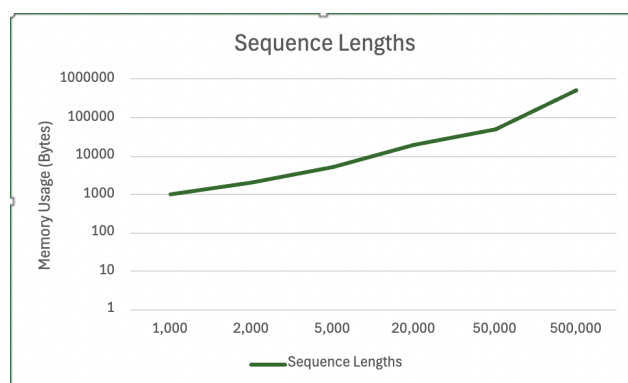


Figure 1: Benchmarking to show that the memory usage is linear to the size of the inputs (the sequences)